

Bases de Datos a Gran Escala

Sesión 4 · Bases de datos basadas en documentos

Diego Sevilla Ruiz

Departamento de Ingeniería y Tecnología de Computadores

Facultad de Informática · Universidad de Murcia

dsevilla@um.es

Introducción a las bases de datos de documentos

Bases de datos documentales

- Organizadas, en general, como un conjunto de **colecciones** que contienen **documentos**:
 - Tablas relacionales ↔ colecciones
 - Filas (*rows*) ↔ documentos
 - No restringen la estructura de cada documento y no tienen esquema
- Cada colección es un *array* donde cada documento tiene asociado un identificador
- La base de datos puede ver el contenido del documento, y utilizar su información como parte de las búsquedas y actualizaciones
- Documentos ⇒ formatos jerárquicos tipo JSON o XML

Bases de datos documentales (ii)

- No utilizan SQL y tienen su propio lenguaje de consulta
- Utilizaban tradicionalmente **Map-Reduce** para cálculos distribuidos; hoy es una técnica histórica, sustituida por el *framework* de agregación
- Algunas implementan otros lenguajes de consulta y procesado, como N1QL (Couchbase) y *Aggregation Framework* (MongoDB)
- Bases de datos de documentos: Couchbase, MongoDB, OrientDB

Modelado de bases de datos de documentos

Modelado de datos en NoSQL

El modelado de datos debe ser:

- Realizado al mayor nivel de abstracción posible
- Independiente de la tecnología subyacente

Sin embargo, en NoSQL:

- Se tiene que tener en cuenta el diseño **distribuido**
- **Optimización guiada por las consultas**

Modelado de datos en NoSQL (ii)

Con respecto al modelo de datos:

- Se mantienen los conceptos de entidad, relación, cardinalidades, etc.
- El modelado relacional se centra en especificar **qué datos tenemos y podemos ofrecer**
- El modelo NoSQL se centra en **optimizar qué consultas vamos a servir**
- Es «barato» **duplicar (desnormalizar)** los datos si con ello se consigue **mayor eficiencia de acceso**
- **Agregar o referenciar** —la decisión que planteamos en la sesión 1— según qué consultas queramos tener optimizadas

Representación de un CV como tablas

Kleppmann, 2016. *Designing Data-Intensive Applications*

<http://www.linkedin.com/in/williamhgates>

**Bill Gates**
Greater Seattle Area | Philanthropy

Summary
Co-chair of the Bill & Melinda Gates Foundation. Chairman, Microsoft Corporation. Voracious reader. Avid traveler. Active blogger.

Experience
Co-chair • Bill & Melinda Gates Foundation
2000 – Present
Co-founder, Chairman • Microsoft
1975 – Present

Education
Harvard University
1973 – 1975
Lakeside School, Seattle

Contact Info
Blog: thegatesnotes.com
Twitter: @BillGates

users table			
user_id	first_name	last_name	summary
251	Bill	Gates	Co-chair of ... blogger.
region_id	industry_id	photo_id	
us:91	131	57817532	

regions table	
id	region_name
us:7	Greater Boston Area
us:91	Greater Seattle Area

industries table	
id	industry_name
43	Financial Services
48	Construction
131	Philanthropy

positions table			
id	user_id	job_title	organization
458	251	Co-chair	Bill & Melinda Gates F...
457	251	Co-founder, Chairman	Microsoft

education table				
id	user_id	school_name	start	end
807	251	Harvard University	1973	1975
806	251	Lakeside School, Seattle	NULL	NULL

contact_info table			
id	user_id	type	url
155	251	blog	http://thegatesnotes.com
156	251	twitter	http://twitter.com/BillGates

Representación de relaciones uno a muchos

Las relaciones uno a muchos (por ejemplo, en el CV: `positions`) en el modelo relacional:

- Normalización usando varias tablas (`Positions` con `user_id`)
 - Necesidad de más de una tabla
 - Necesidad de usar `JOIN` $\Rightarrow$ ineficiencia
- Algunos SGBDR ofrecen la posibilidad de tener tipos de datos estructurados y campos XML o JSON (p. ej. PostgreSQL)
 - Usualmente no se pueden usar para buscar dentro
 - No son estándar

CV como un documento

```
{
  "user_id": 251,
  "first_name": "Bill",
  "last_name": "Gates",
  "summary": "Co-chair of the Bill & Melinda Gates... Active blogger.",
  "region_id": "us:91",
  "industry_id": 131,
  "photo_url": "/p/7/000/253/05b/308dd6e.jpg",
  "positions": [
    { "job_title": "Co-chair", "organization": "Bill & Melinda Gates Foundation" },
    { "job_title": "Co-founder, Chairman", "organization": "Microsoft" }],
  "education": [
    { "school_name": "Harvard University", "start": 1973, "end": 1975 },
    { "school_name": "Lakeside School, Seattle", "start": null, "end": null }],
  "contact_info": {
    "blog": "http://thegatesnotes.com",
    "twitter": "http://twitter.com/BillGates"
  }
}
```

CV como un árbol (equivalente)



Representación de relaciones: modelo de documentos

- **Modelo de documentos** → analogía del **array/mapa gigante**
- **Conjunto de documentos** (objetos complejos):
 - Un **identificador único**, campo *id*
 - Búsqueda aleatoria eficiente por clave (**referencia**)
 - Estructura jerárquica de subdocumentos contenidos → **agregación**

Más flexibilidad que el modelo relacional: elección entre *referencia* y *agregación*

Uno a muchos (ii) — NoSQL

Relaciones **uno a muchos** (`positions`):

- **Opción 1:** agregando la tabla `positions`
- **Opción 2:** convertir las empresas en entidades y utilizar una *referencia*

¡Modelado guiado por el acceso a datos!

- Si los elementos «muchos» tienen una estructura sencilla ⇒ **Opción 1**
- Si los elementos «muchos» se recuperan usualmente **junto con** el elemento «uno» ⇒ **Opción 1**
- Si los elementos «muchos» son relativamente grandes, o se recuperan siempre por separado ⇒ **Opción 2**

Muchos a uno y muchos a muchos

- Ejemplos: personas que viven en una región; preguntas que refieren a *tags*
- El modelo de documentos no aporta ventajas frente al modelo relacional
- Al haber muchas entidades que refieren a otra entidad, la *agregación* daría lugar a mucha **duplicación** (y a problemas de sincronización)
- ⇒ **Referencias** (sobre el `id`), similar a una FK en el modelo relacional
 - **Sin embargo**, al no haber `JOIN`s, la aplicación tiene que hacer más de una petición a la BD

Muchos a muchos — referencia



Introducción a MongoDB

Introducción a MongoDB

- Base de datos documental
- Basada en una representación binaria de JSON, llamada BSON
- Cada base de datos se divide en un conjunto de *colecciones*
- Una colección es un conjunto arbitrariamente grande de documentos
- Cada documento contiene un campo especial `_id`, que guarda un objeto de tipo especial `ObjectID` de BSON
- Permite la definición de bases de datos, colecciones (tablas *hash* de documentos) y subcolecciones
- Permite búsquedas secuenciales, programas MapReduce y un API propietario de consulta (llamado *aggregate*)
- Ofrece un *shell* que acepta JavaScript, y clientes en muchos lenguajes de programación

Uso básico de MongoDB

El *shell* acepta código JavaScript. Existen una serie de objetos predefinidos:

```
show databases;  
use <database>;  
db.<collection>; // Permite acceder a la colección
```

Sólo nombrando las colecciones se puede acceder a ellas para consulta y actualización. Inserción de documentos:

```
db.colección.insert({< Objeto JSON >});
```

Actualización y consulta

Se pueden obtener los documentos de la base de datos con:

```
var objeto = db.colección.findOne({ atributos : valores})  
// ó encontrar todos los que cumplen la condición:  
var objs = db.colección.find({ atributos : valores})
```

Y actualizarlos:

```
db.colección.update(  
  { atributos: valores}, // búsqueda  
  objeto);               // nuevo objeto
```

La selección del objeto se suele hacer por su `_id`, que es único

Actualización: modificadores

```
db.analytics.update({"url" : "www.example.com"},  
                    {"$inc" : {"pageviews" : 1}})
```

También `$set`, `$unset`, `$push` (para *arrays*), `$pull` (para eliminar elementos de un *array* que cumplen un criterio)

Consulta

Las consultas se hacen con `find`:

```
db.users.find({<criterio>},  
              {"username" : 1, "_id" : 0})  
  
// =>  
{  
  "username" : "joe",  
}
```

Y se pueden utilizar condicionantes para la búsqueda:

```
db.users.find({"age": {"$gte": 18, "$lte": 30}});  
db.users.find({"$or": [ {"age": {"$gte": 18} },  
                        {"permission": true} ] }));
```

Lo mismo, con `pymongo` (async): un adelanto de la práctica

Esto es literalmente lo primero que ejecutaremos en la sesión de prácticas, sobre una colección `personas` de prueba. La conexión se abre con `AsyncMongoClient` y cada operación lleva `await`:

```
personas: AsyncCollection = db.personas
await personas.insert_one({"nombre": "Juan", "edad": 25})
await personas.insert_one({"nombre": "Diego", "apellido": "Sevilla", "edad": 30})
await personas.find_one({"nombre": "Diego"})
```

```
await personas.update_one({"nombre": "Diego"}, {"$set": {"edad": 35}})
await personas.update_many(
    {"edad": {"$gte": 18}}, {"$set": {"mayor_de_edad": True}}
)
```

Consultas MapReduce (histórico)

MongoDB lo ofrecía antes de tener un *framework* de agregación propio; hoy está desaprobado a favor de `aggregate()` (más adelante en esta sesión), pero conviene reconocerlo si aparece en código antiguo

```
db.colección.mapReduce(<función map>, <función reduce>, <opciones>)
```

```
var thm_map = function () {  
    var date = new ISODate(this.CreationDate);  
    var dayOfWeek = date.getDay();  
    var hour = date.getHours();  
  
    emit( dayOfWeek + '-' + hour, 1);  
};  
  
var thm_reduce = function (key, values) {  
    return Array.sum(values);  
}
```

Uso de MongoDB desde `pymongo`

En las prácticas usamos siempre el cliente `async` (`AsyncMongoClient`):
toda operación sobre una colección lleva `await` delante

Métodos de búsqueda

El método de búsqueda principal es `find()`, que tiene muchas opciones. En general permite especificar:

- El filtro de búsqueda
- Ordenación de resultados por algún campo
- Proyección, para no obtener todos los campos del documento
- Número de resultados máximo (*limit*)
- Número de elementos iniciales a ignorar (*skip*)
- El tamaño del *batch*

Como la variabilidad es muy grande, se ven ejemplos en la hoja Jupyter de prácticas y en la documentación

Métodos de búsqueda: calificadores

`find()` admite calificadores complejos que unen «objetos», no valores:

- `$and`, `$or`, `$not`

Y otros que se refieren a valores:

- `$lt` (menor), `$lte` (menor o igual), `$gt` (mayor), `$gte` (mayor o igual), `$regex` (expresión regular)

```
await posts.find_one({"body": {"$regex" : "[Mm]ongo"}})
```

Condición compuesta (limitando la salida a 10 documentos):

```
await posts.find({"$and": [{"PostTypeId": 2},  
                           {"Id" : {"$gte" : 100}} ]}).limit(10).to_list()
```

Métodos de búsqueda: ejemplos introductorios

Encontrar el primer documento en la colección `customers` :

```
client = AsyncMongoClient("mongodb://localhost:27017/")
mydb = client["mydatabase"]
mycol = mydb["customers"]
x = await mycol.find_one()
```

Encontrar todos los documentos en la colección `customers` :

```
async for x in mycol.find():
    print(x)
```

Métodos de búsqueda: proyección

También podemos indicar qué campos queremos obtener de una determinada búsqueda:

```
async for x in mycol.find({}, { "_id": 0, "name": 1, "address": 1 }):  
    print(x)
```

Ejemplos básicos

Dada la siguiente colección:

```
{ "_id": "manzanas", "cantidad": 5 }  
{ "_id": "plátanos", "cantidad": 7 }  
{ "_id": "naranjas", "cantidad": 8 }  
{ "_id": "aguacate", "cantidad": 14 }
```

Documentos con `cantidad` superior a 4:

```
db.collection.find( { "cantidad": { "$gt": 4 } } )
```

Filtrado por rango de fechas

Suponiendo una colección `bios` cuyos documentos tienen esta forma:

```
{
  "_id": <valor>,
  "nombre": { "nombre_propio": <string>, "primer_apellido": <string>},
  "nacimiento": <ISODate>,
  "muerte": <ISODate>,
  "profesion": <valor>,
  "trabajos": [<string>, ...]
}
```

```
from datetime import datetime
db.bios.find( { "nacimiento": { "$gt": datetime(1940,1,1),
  "$lt": datetime(1960,1,1) } } )
```

El operador \$in

Permite buscar documentos con atributos dentro de una lista de valores:

```
db.bios.find(  
  { "profesion": { "$in": [ 'jedi', 'samurai' ] } }  
)
```

Campos anidados y arrays

También podemos realizar búsquedas en campos anidados:

```
db.bios.find( { "nombre.primer_apellido": "Skywalker" } )  
db.bios.find(  
  { "nombre": { "nombre_propio": "Akira", "primer_apellido": "Kurosawa" } }  
)
```

Las búsquedas pueden realizarse también sobre campos cuyos valores son *arrays*:

```
db.bios.find({"trabajos": "star_wars"})  
db.bios.find({"trabajos": { "$in": [ "star wars", "alien" ] } })  
db.bios.find({"trabajos": { "$all": [ "star wars", "the clone wars" ] } })  
db.bios.find({"trabajos": { "$size": 4 } }) # 'trabajos' debe tener 4 elementos
```

\$not, \$and, \$or

\$not es útil para buscar elementos que NO cumplan una condición:

```
# Documentos cuyo precio no sea superior a 1,99€ o que no tengan campo "precio"
db.inventory.find( { "precio": { "$not": { "$gt": 1.99 } } } )
```

\$and, \$or y \$not pueden combinarse para condiciones complejas:

```
db.inventory.find( {
  "$and": [
    {"$or":[{"cantidad":{"$lt":10}}, {"cantidad":{"$gt":50} }]},
    {"$or":[{"vendido": true}, {"precio": {"$lt": 5}}]}
  ]
} )
```

¿Qué documentos devolverá la consulta de arriba?

Métodos de inserción y actualización

`pymongo` ofrece métodos para inserción y actualización:

- `insert_one()` , `insert_many()` (*batch*)
- `update_one()` : actualiza un objeto con nuevos campos. Se crea si `upsert=True`
- `update_many()` : aplica nuevos valores calculados a un conjunto de objetos

Inserción: ejemplos básicos

Al insertar un elemento podemos obtener su `_id`:

```
micoleccion = db["clientes"]
midict = { "nombre": "Luke", "direccion": "Tatooine, 83" }
elem = mycol.insert_one(midict)
print(elem.inserted_id)
```

Para insertar un conjunto de documentos a la vez, se incluyen en un *array*:

```
milista = [ { "nombre": "Amy", "direccion": "Apple st 652"},
             { "nombre": "Hannah", "direccion": "Mountain 21"},
             { "nombre": "Michael", "direccion": "Valley 345"} ]
elems = micoleccion.insert_many(milista)
print(elems.inserted_ids)
```

Actualización: ejemplos básicos

Debemos especificar un criterio de búsqueda y los campos y valores a actualizar:

```
micoleccion = db["clientes"]
mifiltro = { "direccion": "Valley 345" }
nuevosvalores = { "$set": { "direccion": "Rue del percebe, 13" } }
micoleccion.update_one(mifiltro, nuevosvalores)
```

Para actualizar múltiples documentos, la estructura es la misma:

```
mifiltro = { "direccion": { "$regex": "^S" } }
nuevosvalores = { "$set": { "nombre": "Rey" } }
x = micoleccion.update_many(mifiltro, nuevosvalores)
print(x.modified_count, "documentos actualizados.")
```

Índices, `explain()`

- La llamada `.explain()` sobre cualquier búsqueda muestra el plan de ejecución
- Se puede crear un índice si la búsqueda por ese campo va a ser crítica
- Se pueden crear más índices:
 - `ASCENDING`
 - `DESCENDING`
 - `HASHED`
 - y geoespaciales

`explain()`: leer `totalDocsExamined`

Antes de crear el índice, cada búsqueda por `Reputation` recorre toda la colección:

```
plan: dict = await users.find({"Reputation": 100}).explain()  
plan["executionStats"]["totalDocsExamined"] # tantos como usuarios haya
```

Después de crear el índice, la misma búsqueda examina sólo lo necesario:

```
await users.create_index([("Reputation", 1)])  
plan = await users.find({"Reputation": 100}).explain()  
plan["executionStats"]["totalDocsExamined"] # sólo los que tienen Reputation = 100
```

En la práctica compararemos además un índice **compuesto** (`Reputation, Views`)

Colecciones tipadas con TypedDict

`pymongo` deja anotar una colección con el tipo de sus documentos, con la ventaja habitual del tipado: autocompletado y comprobación estática, sin cambiar ninguna consulta:

```
class User(TypedDict):
    Id: int
    DisplayName: str
    Reputation: int
    WebsiteUrl: NotRequired[str] # puede faltar: no todos los usuarios lo rellenan

users_t: AsyncCollection[User] = db["users"]
user: User | None = await users_t.find_one({"Id": 22656})
```

`NotRequired` marca los campos que pueden **faltar** del documento, que en MongoDB no es lo mismo que valer `None`: es la misma distinción de la sesión 1 entre ausencia y `NULL`

Map-Reduce

Collection
↓
db.orders.mapReduce(
 map → function() { emit(this.cust_id, this.amount); },
 reduce → function(key, values) { return Array.sum(values) },
 {
 query → { status: "A" },
 output → "order_totals"
 }
)



Map-Reduce en `pymongo` (histórico)

- La función `map_reduce()` de cada colección permite ejecutar procesamientos que guardan los resultados en otra colección o los devuelven en vivo
- Como el código se ejecuta dentro del motor de la base de datos, hay que enviarlo en JavaScript
- Ejemplo: histograma del tamaño del texto de los *posts*
- **No se usa en las prácticas:** el mismo cálculo se hace con el *framework* de agregación, que veremos a continuación y es lo que usaremos siempre

Map-Reduce en pymongo (ii)

```
from bson.code import Code
map = Code(
    '''function () {
        if ('Body' in this)
            emit(this.Body.length, 1)
        else
            emit(0,1)
    }''')
reduce = Code(
    '''function (key, values) {
        return Array.sum(values);
    }''')
results = posts.map_reduce(map, reduce, "myresults")
```

Ejercicio: ¿y la media cómo sería?

Framework de agregación



Framework de agregación

- Etapas (*stages*) del *pipeline*: documentación de agregación
- Operadores dentro de las etapas: documentación de operadores

```
await posts.aggregate( [  
  {'$project' : { 'Id' : 1 }},  
  {'$limit' : 20}  
)
```

Las operaciones de agregación pueden convertirse en complejas, con muchos operadores donde elegir: algunos poco frecuentes como

`$ifNull` y `$strLenBytes`, y otros más complejos como `$expr` y `$cond`

Framework de agregación: ejemplo

```
by_length = await posts.aggregate( [
  {'$project': {
    'Body' : {'$ifNull' : ['$Body', '']}
  }},
  {'$project' : {
    'id' : {'$strLenBytes' : '$Body'},
    'value' : {'$literal' : 1}
  }},
  {'$group' : {
    '_id' : '$id',
    'count' : {'$sum' : '$value'}
  }},
  {'$sort' : { '_id' : 1}}
])
```

Filtrar y unir colecciones

Se pueden añadir etapas; por ejemplo, **filtrar posts**:

```
{ '$match': { 'Body' : { '$regex': 'HBase' } } }
```

`$lookup` permite el acceso a otra (o a la misma) colección: equivalente a un **JOIN**. Listar los *posts* con `Score` mayor o igual a 40 junto con el usuario que lo ha hecho:

```
await posts.aggregate( [  
  { '$match': { 'Score' : { '$gte': 40 } } },  
  { '$lookup': {  
    'from': "users",  
    'localField': "OwnerUserId",  
    'foreignField': "Id",  
    'as': "owner"  
  } }  
])
```

\$lookup: resultado y \$unwind

El resultado devuelve el campo `owner` relleno con cada usuario, en un *array*:

```
{
  'Id' : xxx, 'Title': ...,
  'owner': [ { 'Id': yyy, 'DisplayName': "zzz" ... }, ... ], ...
}
```

Esto es así porque MongoDB no sabe si el `foreignField` generará más de un elemento. La etapa `$unwind` convierte el *array* en varios documentos, uno por elemento:

```
... (resto del pipeline)
{ "$unwind" : "owner" },
...
```

\$unwind: resultado

```
{  
  'Id' : xxx, 'Title': ...,  
  'owner': { 'Id': yyy, 'DisplayName': "zzz" ... }, ...  
}
```

Ha desaparecido el *array*: se genera un documento igual para cada valor que `owner` tuviera

Framework de agregación — ejemplo (i)

Ordenar por fecha ascendente los *posts* con el *tag* «java», de más antiguo a más reciente:

```
posts = db["posts"]
pipeline = [
    { "$match": { "Tags": { "$regex": "<java>" } } },
    { "$sort": { "CreationDate": pymongo.ASCENDING } }
]
results = await posts.aggregate(pipeline)
```

Si ahora quisiéramos limitar la salida a un único documento:

```
pipeline = [
    { "$match": { "Tags": { "$regex": "<java>" } } },
    { "$sort": { "CreationDate": pymongo.ASCENDING } },
    { "$limit": 1 }
]
```

Framework de agregación — ejemplo (ii)

Se puede usar también `$group` para ver los *posts* de java de cada año:

```
stage_match_tag = {
    "$match": { "Tags": {"$regex": "<java>"} }
}
stage_group_year = {
    "$group": {
        "_id": {"$year" : "$CreationDate"},
        "post_count": { "$sum": 1 } } }
stage_sort_year_ascending = {
    "$sort": {"_id": pymongo.ASCENDING}
}
pipeline = [stage_match_tag, stage_group_year, stage_sort_year_ascending]
results = await posts.aggregate(pipeline)
```